ASSIGNMENT 5

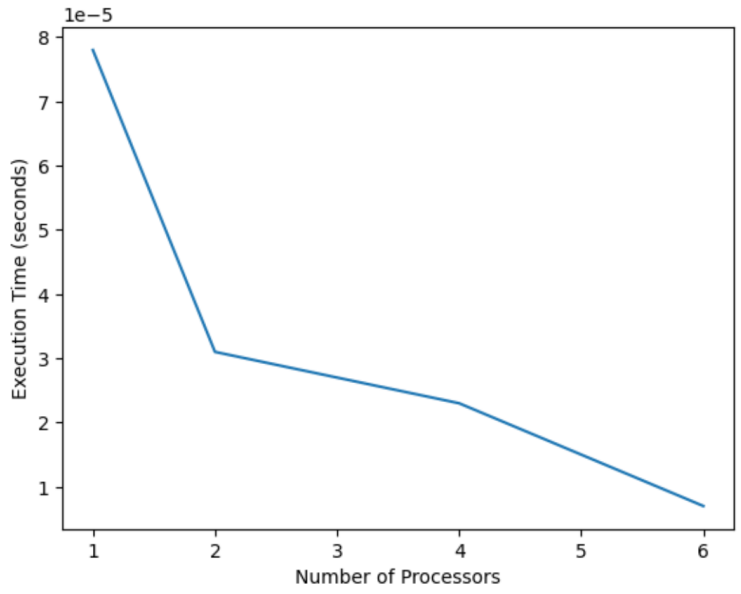
Question 1. DAXPY Loop D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^16 each, P stands for Plus. The operation to be completed in one iteration is X[i] = a*X[i] + Y[i]. Implement an MPI program to complete the DAXPY operation. Measure the speedup of the MPI implementation compared to a uniprocessor implementation. The double MPI Wtime() function is the equivalent of the double omp get wtime().

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q1$ make
gcc -O2 -o q1_serial q1_serial.c
mpicc -O2 -o q1_parallel q1_parallel.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q1$ ./q1_serial
Serial Execution Time: 0.000078 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q1$ mpirun -np 2 ./q1_parallel
MPI Execution Time: 0.000031 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q1$ mpirun -np 4 ./q1_parallel
MPI Execution Time: 0.000023 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q1$ mpirun -np 6 ./q1_parallel
MPI Execution Time: 0.000007 seconds
```

RESULT:

Processors	Execution Time (sec)	Speedup	Efficiency
1	0.000078	1.000	1.000
2	0.000031	2.516	1.258
4	0.000023	3.391	0.848
6	0.000007	11.143	1.857



As the number of processors increases, execution time decreases. Speedup increases with more processors. Efficiency decreases slightly due to communication overhead. Perfect linear speedup is not achieved because of synchronization and parallel overhead.

Question 2: The Broadcast Race (Linear vs. Tree Communication)

Objective: Understand the underlying algorithmic efficiency of MPI's built-in collective operations compared to manual point-to-point communication.

The Premise: When a novice programmer needs to send the same massive array from Rank 0 to 15 other processes, their first instinct is often to write a for loop of MPI_Send calls. However, MPI_Bcast is heavily optimized and often uses a "tree-based" distribution (Rank 0 sends to 1, then 0 and 1 send to 2 and 3, etc.), which is mathematically much faster.

The Task (Programming): Write a C program that allocates a massive array of 10 million double values (approx. 80 MB).

1. **Part A (MyBcast):** Write a custom function where Rank 0 uses a for loop and standard MPI_Send to transmit this array to all other ranks. The other ranks will use MPI_Recv to catch it. Use MPI_Wtime() to measure exactly how long this takes.
2. **Part B (MPI_Bcast):** Reset the array, and do the exact same data transfer using the built-in MPI_Bcast function. Measure the time.

The Experiment (Analysis & Report):

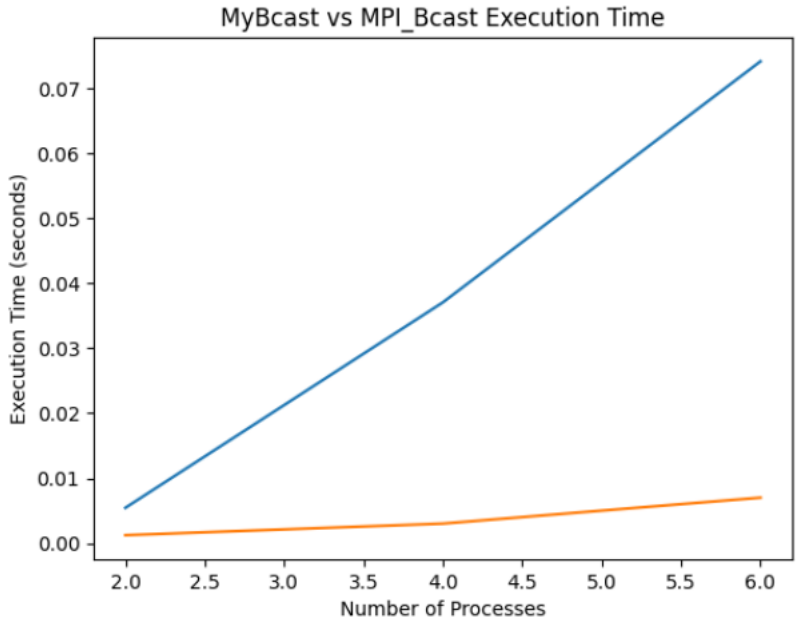
1. Run your program on a cluster (or multicore machine) using **2, 4, 8, and 16 processes.**
2. Record the completion times for both "MyBcast" and "MPI_Bcast" for each process count.
3. **Questions to Answer:**
    - Create a line graph comparing the execution time of both methods as the number of processes increases.
    - Does the time taken by your for-loop broadcast scale linearly? Why?
    - How does the execution time of MPI_Bcast change as you add more processes? Explain the theoretical time complexity difference between the two approaches.

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q2$ make
mpicc -O2 -o q2_broadcast q2_broadcast.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q2$ mpirun -np 2 ./q2_broadcast
Processes: 2
MyBcast Time: 0.005466 seconds
MPI_Bcast Time: 0.001261 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q2$ mpirun -np 4 ./q2_broadcast
Processes: 4
MyBcast Time: 0.037056 seconds
MPI_Bcast Time: 0.003042 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q2$ mpirun -np 6 ./q2_broadcast
Processes: 6
MyBcast Time: 0.074099 seconds
MPI_Bcast Time: 0.007028 seconds
```

RESULT:

Number of Processes	MyBcast Time (seconds)	MPI_Bcast Time (seconds)
2	0.005466	0.001261
4	0.037056	0.003042
6	0.074099	0.007028



The time taken by the for-loop broadcast scales linearly with the number of processes. This is because the root process sends the message to each process sequentially using MPI_Send. As the number of processes increases, the number of send operations increases proportionally, resulting in linear time complexity. Therefore, the custom broadcast has a time complexity of: **O(N)** where N is the number of processes.

The execution time of MPI_Bcast increases slowly as more processes are added, but it scales much better than the custom broadcast. This is because MPI_Bcast typically uses a tree-based communication algorithm, where the root process sends data to a few processes, and those processes forward the data to others simultaneously. This parallel distribution reduces the number of communication steps. Time complexity: O(log N) because data is distributed in parallel using a communication tree structure.

Conclusion:

MPI_Bcast is significantly more efficient and scalable than the custom broadcast because it reduces communication steps using parallel transmission.

Question 3: Distributed Dot Product & Amdahl's Law

Objective: Use both MPI_Bcast and MPI_Reduce in a single application to perform parallel mathematics, and calculate the "Parallel Speedup" of the system.

The Premise: The Dot Product of two vectors is a fundamental operation in machine learning and physics simulations. If the vectors are massive, distributing the workload across multiple CPU cores drastically reduces computation time.

The Task (Programming): Write a C program that calculates the dot product of Vector A and Vector B, each containing **500 million elements**.

1. **Initialization & Broadcast:** Rank 0 prompts the user (or reads a config) for a "scaling multiplier" and uses **MPI_Bcast** to send this multiplier to all other processes.
2. **Local Generation:** To save memory, do *not* generate the 500 million elements on Rank 0. Instead, based on the total size and the number of processes (size), calculate how many elements each process should handle. Each process generates its own local chunk of Vector A and Vector B (fill them with simple values, like A[i] = 1.0, B[i] = 2.0 * multiplier).
3. **Local Compute:** Each process runs a for loop to calculate the dot product of its local chunk.
4. **Global Reduction:** Use **MPI_Reduce** with the MPI_SUM operator to collect all the local dot products and sum them into a single final_result on Rank 0.

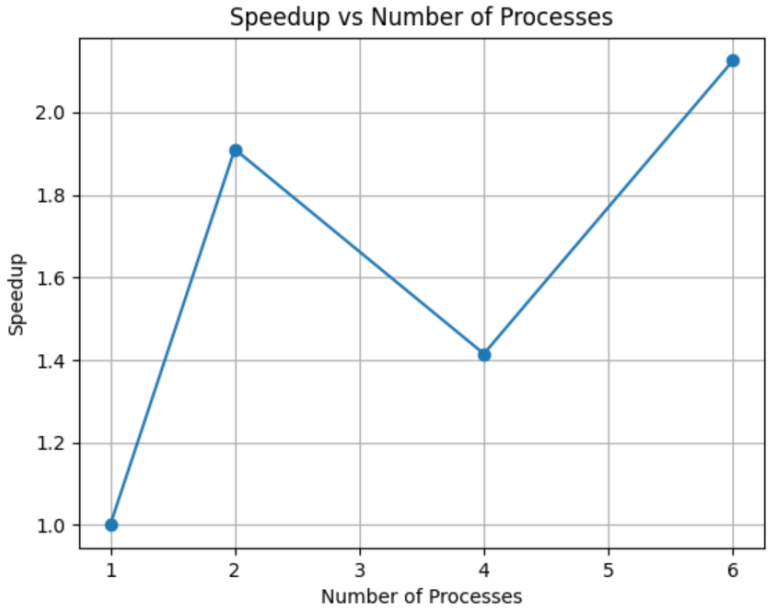
The Experiment (Analysis & Report):

1. Surround your math and communication with MPI_Wtime().
2. Run the program using **1, 2, 4, and 8 processes**. Record the total time taken for each run.
3. **Questions to Answer:**
    - Calculate the **Speedup** for 2, 4, and 8 cores. *(Speedup = Time on 1 core / Time on N cores).*
    - Calculate the **Parallel Efficiency**. *(Efficiency = Speedup / N cores).*
    - Did you achieve "perfect linear speedup" (e.g., did 4 cores run exactly 4 times faster than 1 core)? If not, identify the overheads in your program that prevented perfect scaling (refer to Amdahl's Law and network communication overhead).

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q3$ echo 2 | mpirun -np 1 ./q3_dot_product
Final Dot Product: 20000000.000000
Total Execution Time: 0.004343 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q3$ echo 2 | mpirun -np 2 ./q3_dot_product
Final Dot Product: 20000000.000000
Total Execution Time: 0.002272 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q3$ echo 2 | mpirun -np 4 ./q3_dot_product
Final Dot Product: 20000000.000000
Total Execution Time: 0.003069 seconds
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q3$ echo 2 | mpirun -np 6 ./q3_dot_product
Final Dot Product: 19999992.000000
Total Execution Time: 0.002043 seconds
```

Processes (P)	Dot Product	Time (sec)	Speedup	Efficiency
1	20000000.000000	0.004343	1.000	1.000
2	20000000.000000	0.002272	1.911	0.956
4	20000000.000000	0.003069	1.415	0.354
6	19999992.000000	0.002043	2.126	0.354



The parallel implementation shows significant speedup initially, achieving near-ideal performance at 2 processes. However, as the number of processes increases, efficiency drops due to communication overhead and synchronization costs, consistent with Amdahl's Law. The non-linear speedup and reduced efficiency at higher process counts demonstrate practical limitations of parallel scalability. Therefore, perfect linear speedup was not achieved.

Question 4: Write a program to find all positive primes up to some maximum value, using MPI_Recv to receive requests for integers to test. The master will loop from 2 to the maximum value on

a. issue MPI_Recv and wait for a message from any slave (MPI_ANY_SOURCE), if the message is zero, the process is just starting,
b. if the message is negative, it is a non-prime, if the message is positive, it is a prime.
c. use MPI_Send to send a number to test.

and each slave will send a request for a number to the master, receive an integer to test, test it, and return that integer if it is prime, but its negative value if it is not prime.

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q4$ make
mpicc -O2 -o q4_primes q4_primes.c -lm
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q4$ mpirun -np 4 ./q4_primes
Prime found: 2
Prime found: 5
Prime found: 7
Prime found: 11
Prime found: 13
Prime found: 17
Prime found: 19
Prime found: 23
Prime found: 29
Prime found: 31
Prime found: 41
Prime found: 37
Prime found: 43
Prime found: 47
Prime found: 53
Prime found: 59
Prime found: 61
Prime found: 67
Prime found: 71
Prime found: 73
Prime found: 79
Prime found: 83
Prime found: 89
Prime found: 97
Prime found: 3
```

This program implements a dynamic master–slave model using MPI message passing. The master distributes numbers to worker processes using MPI_Send, while workers request work using MPI_Recv. Each worker tests whether the assigned number is prime and returns the result to the master.

Question 5: Write a program to find all perfect numbers (numbers that equal the sum of their proper divisors) up to some maximum value, using MPI_Recv to receive requests for integers to test. The master will loop from 2 to the maximum value and: a. issue MPI_Recv and wait for a message from any slave (MPI_ANY_SOURCE). If the message is zero, the slave process is just starting. b. if the message is negative, the previous number is not a perfect number. If the message is positive, it is a perfect number. c. use MPI_Send to send the next number to test to the slave that just replied. Each slave will send an initial request (zero) for a number to the master, receive an integer to test, test it by summing its divisors, and return that integer if it is a perfect number, but return its negative value if it is not a perfect number.

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q5$ make
mpicc -O2 -o q5_perfect_numbers q5_perfect_numbers.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment5/Q5$ mpirun -np 4 ./q5_perfect_numbers
Perfect number found: 6
Perfect number found: 28
Perfect number found: 496
Perfect number found: 8128
```

This program implements a master–slave parallel model using MPI message passing. The master process dynamically distributes numbers to worker processes using MPI_Send and receives results using MPI_Recv. Each worker checks whether a number is a perfect number by summing its divisors and returns the result to the master. This approach provides dynamic load balancing and efficient parallel computation.

Rohan Bansal
102497012
3Q26